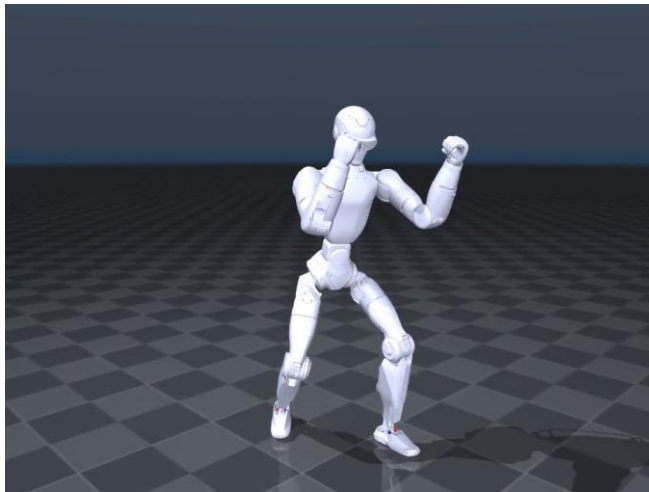


Embodied AI

Setup Guide for GMR

Installing GMR and Adding the R1

September 2026



Shaoqin Li

Applied and Embodied AI Lab
Electrical Engineering Department
The City College of New York

about 30 minutes · 3.6 GB download · do this days before the session

Contents

Why a separate environment	3
1 — The environment and the clone	3
2 — The R1 is not in GMR	3
3 — Get the four support files	4
4 — Create the installer	4
5 — Run it	7
6 — Create the checker and run it.....	7
The motion file.....	10
If you cannot get it installed	11
Troubleshooting.....	11

DO THIS BEFORE THE SESSION

Labs 1–3 you built entirely by hand, from an empty folder. **Lab 4 is different:** it uses a real research repository — **GMR**, General Motion Retargeting — cloned from GitHub, about 3.6 GB with its robot assets and submodule.

There is **no time to install this during the lab**. Budget 30 minutes at home, most of it download. When you are done, run the checker at the bottom of this page and **bring its verdict to the session**. If it says **READY**, you are finished.

Everything here is typed in the same Ubuntu terminal you used for Labs 1–3. You need Lab 1 finished, because step 5 reuses the 43 robot meshes you downloaded there.

Why a separate environment

GMR needs **mink** (an inverse-kinematics solver) and MuJoCo **3.6.0**. Labs 1–3 pin MuJoCo 3.12.0. Installing both into one environment breaks both.

Keep them apart: **gmr** for this lab, **r1lab** for the others. Neither knows the other exists.

1 — The environment and the clone

```
conda create -n gmr python=3.10 -y
conda activate gmr
```

```
cd ~
git clone --recurse-submodules https://github.com/YanjieZe/GMR.git
cd GMR
pip install -e .
```

```
conda install -c conda-forge libstdcxx-ng -y
```

The last line is Linux-only — skip it on macOS.

--recurse-submodules is not optional. GMR pulls **mujoco_menagerie** as a submodule. Without it you get a clone that imports fine and then fails on a missing model file. If you forgot:

```
git submodule update --init --recursive
```

2 — The R1 is not in GMR

Stock GMR ships eleven robots. **The Unitree R1 is not one of them.** Check for yourself, before you add it:

```
python -c "from general_motion_retargeting import IK_CONFIG_DICT; \
print({k: 'unitree_r1' in v for k, v in IK_CONFIG_DICT.items()})"
```

Expected — every single one `False`:

```
{'smplx': False, 'bvh_lafan1': False, 'bvh_nokov': False, 'bvh_xsens': False,
'fbx': False, 'fbx_offline': False, 'xrobot': False, 'xsens_mvn': False}
```

That is a correct, working GMR install that cannot retarget onto an R1 at all. Support for the R1 was written by this project, and adding a robot to a retargeting tool is exactly four things:

<code>assets/unitree_r1/r1_mocap.xml + r1_scene.xml</code>	the robot model
<code>assets/unitree_r1/assets/</code>	its 43 shapes — the same STL files you downloaded in Lab 1
<code>ik_configs/bvh_xsens_to_r1.json + smplx_to_r1.json</code>	Xsens and SMPL-X → R1 mappings
5 entries in <code>params.py</code>	model path, 2 IK configs, root body, camera distance

The IK configs are the actual contribution. Each is a JSON file saying which human body part drives which robot body, and how much each one matters. That mapping is what a new robot needs, and writing one is the work. The rest is plumbing.

3 — Get the four support files

Your instructor gives you `r1_gmr_support.tar.gz` — 8 KB, the two model files and the two IK configs. They are text; you could read every line of them.

```
mkdir -p ~/r1_lab/exp4
cd ~/r1_lab/exp4
tar -xzf ~/Downloads/r1_gmr_support.tar.gz
ls support
```

Expected:

```
bvh_xsens_to_r1.json  r1_mocap.xml  r1_scene.xml  smplx_to_r1.json
```

4 — Create the installer

The installer is not hidden from you either. Create `install_r1_into_gmr.py` in `~/r1_lab/exp4`:

```
"""Add the Unitree R1 to a stock GMR clone -- Lab 4, Part 0.

GMR ships eleven robots. The R1 is not one of them: support for it was written
by this project, and a clone straight from GitHub cannot retarget onto an R1 at
```

all. Adding a robot to a retargeting tool is four things, and this script does all four, printing each one:

1. the robot model assets/unitree_r1/r1_mocap.xml + r1_scene.xml
2. its 43 shapes the same STL files you downloaded in Lab 1
3. two IK configs bvh_xsens_to_r1.json, smplx_to_r1.json
4. five entries in params.py model path, 2 IK configs, root body, camera

Run it once, from the exp4 folder, with the `gmr` environment active:

```
python install_r1_into_gmr.py
```

It is safe to run twice -- anything already in place is reported and skipped.

```
"""
```

```
import argparse
import os
import shutil
import sys
```

```
HERE = os.path.dirname(os.path.abspath(__file__))
DEFAULT_GMR = os.path.join(os.path.expanduser("~"), "GMR")
DEFAULT_MESHES = os.path.join(os.path.expanduser("~"), "r1_lab", "exp1",
                               "model", "assets")
```

```
# (file in support/, destination relative to the GMR clone)
```

```
FILES = [
    ("r1_mocap.xml", os.path.join("assets", "unitree_r1", "r1_mocap.xml")),
    ("r1_scene.xml", os.path.join("assets", "unitree_r1", "r1_scene.xml")),
    ("bvh_xsens_to_r1.json", os.path.join("general_motion_retargeting",
                                           "ik_configs", "bvh_xsens_to_r1.json")),
    ("smplx_to_r1.json", os.path.join("general_motion_retargeting",
                                      "ik_configs", "smplx_to_r1.json")),
]
```

```
# (anchor line already in params.py, the line to insert after it)
```

```
PARAMS = [
    ('    "unitree_h1": ASSET_ROOT / "unitree_h1" / "h1.xml",' ,
     '    "unitree_r1": ASSET_ROOT / "unitree_r1" / "r1_scene.xml",' ),
    ('    "unitree_h1": IK_CONFIG_ROOT / "smplx_to_h1.json",' ,
     '    "unitree_r1": IK_CONFIG_ROOT / "smplx_to_r1.json",' ),
    ('    "unitree_h1_2": IK_CONFIG_ROOT / "bvh_xsens_to_h1_2.json",' ,
     '    "unitree_r1": IK_CONFIG_ROOT / "bvh_xsens_to_r1.json",' ),
    ('    "unitree_h1_2": "pelvis",' , '    "unitree_r1": "pelvis",' ),
    ('    "unitree_h1_2": 3.0,' , '    "unitree_r1": 2.0,' ),
]
```

```
def main():
```

```
    ap = argparse.ArgumentParser()
    ap.add_argument("--gmr", default=DEFAULT_GMR, help="your GMR clone")
    ap.add_argument("--meshes", default=DEFAULT_MESHES,
                    help="the 43 STL files from Lab 1")
    ap.add_argument("--support", default=os.path.join(HERE, "support"),
                    help="the four R1 support files")
```

```

a = ap.parse_args()

if not os.path.isdir(a.gmr):
    raise SystemExit("no GMR clone at %s -- see the setup guide" % a.gmr)
print("GMR clone:", a.gmr)

# 1 + 3. the four text files
for name, rel in FILES:
    src = os.path.join(a.support, name)
    if not os.path.exists(src):
        raise SystemExit("missing %s -- did the support download land?" % src)
    dst = os.path.join(a.gmr, rel)
    os.makedirs(os.path.dirname(dst), exist_ok=True)
    same = os.path.exists(dst) and open(dst, "rb").read() == open(src, "rb").read()
    shutil.copyfile(src, dst)
    print(" %-24s -> %s%s" % (name, rel, " (already there)" if same else ""))

# 2. the shapes -- the same STLs Lab 1 pulled from Unitree's repository
dst_assets = os.path.join(a.gmr, "assets", "unitree_r1", "assets")
if not os.path.isdir(a.meshes):
    raise SystemExit("no meshes at %s -- pass --meshes" % a.meshes)
os.makedirs(dst_assets, exist_ok=True)
n = 0
for f in sorted(os.listdir(a.meshes)):
    shutil.copyfile(os.path.join(a.meshes, f), os.path.join(dst_assets, f))
    n += 1
print(" %d meshes copied from %s" % (n, a.meshes))

# 4. five entries in params.py
params = os.path.join(a.gmr, "general_motion_retargeting", "params.py")
text = open(params).read()
added = 0
for anchor, line in PARAMS:
    if line in text:
        continue
    if anchor not in text:
        raise SystemExit("cannot find the anchor line in params.py:\n %s\n"
                        "GMR has changed upstream; patch it by hand." % anchor)
    text = text.replace(anchor, anchor + "\n" + line, 1)
    added += 1
if added:
    shutil.copyfile(params, params + ".before_r1")
    open(params, "w").write(text)
print(" params.py: %d entr%s added%s" % (added, "y" if added == 1 else "ies",
                                         " (backup: params.py.before_r1)"
                                         if added else " -- already patched"))

# verify, the only line that matters
sys.path.insert(0, a.gmr)
from general_motion_retargeting import IK_CONFIG_DICT
have = {k: "unitree_r1" in v for k, v in IK_CONFIG_DICT.items()}
print("\n R1 accepted from:", ", ".join(k for k, v in have.items() if v) or "nothing")
assert have.get("bvh_xsens") and have.get("smplx"), have
print(" OK -- the R1 is installed in GMR")

```

```
if __name__ == "__main__":
    main()
```

5 — Run it

```
python install_r1_into_gmr.py
```

Expected:

```
GMR clone: /home/YOURNAME/GMR
r1_mocap.xml      -> assets/unitree_r1/r1_mocap.xml
r1_scene.xml      -> assets/unitree_r1/r1_scene.xml
bvh_xsens_to_r1.json -> general_motion_retargeting/ik_configs/bvh_xsens_to_r1.json
smplx_to_r1.json  -> general_motion_retargeting/ik_configs/smplx_to_r1.json
43 meshes copied from /home/YOURNAME/r1_lab/exp1/model/assets

params.py: 5 entries added (backup: params.py.before_r1)

R1 accepted from: smplx, bvh_xsens
OK -- the R1 is installed in GMR
```

The 43 meshes come from your Lab 1 folder. GMR's R1 uses the same STL files you pulled out of Unitree's repository in Lab 1 Part 2 — the same shapes your standing robot has been made of all along. If Lab 1's folder is gone, pass `--meshes` pointing at any copy of them.

`xrobotoolkit_sdk not found, skip for now.` prints on every single GMR command. It is a VR streaming dependency nothing here uses. **Ignore it. It is not an error.**

6 — Create the checker and run it

Create `check_setup.py` in `~/r1_lab/exp4`:

```
"""Am I ready for Lab 4? Run this BEFORE the session.

conda activate gmr
python check_setup.py

It prints one line per requirement and a single verdict at the end. Bring that
verdict to the lab. If it says READY, you are done -- nothing else to install.

If something fails, the message tells you which command in LAB4_setup_gmr.md
fixes it. Do not wait until the session to find out.
```

```

"""
import importlib, os, sys

OK, BAD, SKIP = " [ok] ", " [--] ", " [..] "
problems = []

def check(label, fn, fix, needs=True):
    # `needs` gates a check on an earlier one. Without this, a wrong conda env
    # makes every downstream check fail and tell you to re-clone the repo --
    # four confident wrong answers instead of the one real cause.
    if not needs:
        print(SKIP + f"{label:38s} skipped -- fix the above first")
        return False
    try:
        detail = fn()
    except Exception as e:
        print(BAD + f"{label:38s} {type(e).__name__}")
        problems.append((label, fix))
        return False
    if detail is False:
        print(BAD + f"{label:38s} not found")
        problems.append((label, fix))
        return False
    print(OK + f"{label:38s} {detail if detail is not True else ''}")
    return True

print("\n" + "=" * 66)
print(" Lab 4 setup check")
print("=" * 66)

check("python 3.10+",
      lambda: f"{sys.version_info.major}.{sys.version_info.minor}" if sys.version_info >= (3, 10)
      else False,
      "conda create -n gmr python=3.10 -y")

check("mujoco",
      lambda: importlib.import_module("mujoco").__version__,
      "pip install -e . (inside the GMR folder)")

check("mink (the IK solver)",
      lambda: importlib.import_module("mink") and True,
      "pip install -e . (inside the GMR folder)")

gmr_ok = check("general_motion_retargeting",
               lambda: importlib.import_module("general_motion_retargeting") and True,
               "you are probably in the wrong environment. Run: conda activate gmr\n"
               " If that is not it: cd into the GMR folder and run pip install -e .")

def r1_model():
    from general_motion_retargeting import ROBOT_XML_DICT
    p = ROBOT_XML_DICT.get("unitree_r1")

```



```

    return str(p).split(os.sep)[-1] if p and os.path.exists(p) else False

check("R1 robot model", r1_model,
      "the R1 assets are missing -- re-clone with --recurse-submodules", gmr_ok)

def r1_configs():
    from general_motion_retargeting import IK_CONFIG_DICT
    have = [k for k, v in IK_CONFIG_DICT.items() if "unitree_r1" in v]
    return ", ".join(have) if "bvh_xsens" in have else False

check("R1 motion mappings", r1_configs,
      "bvh_xsens_to_r1.json is missing from general_motion_retargeting/ik_configs/", gmr_ok)

def the_motion():
    from general_motion_retargeting import ASSET_ROOT
    p = os.path.join(str(ASSET_ROOT), "xsens_bvh_test",
                     "251021_04_boxing_120Hz_cm_3DsMax.bvh")
    return f"{os.path.getsize(p)//1024} KB" if os.path.exists(p) else False

check("the boxing capture", the_motion,
      "assets/xsens_bvh_test/ is missing -- re-clone with --recurse-submodules", gmr_ok)

def menagerie():
    from general_motion_retargeting import ASSET_ROOT
    p = os.path.join(os.path.dirname(str(ASSET_ROOT).rstrip(os.sep)), "mujoco_menagerie")
    return True if os.path.isdir(p) and os.listdir(p) else False

check("mujoco_menagerie submodule", menagerie,
      "git submodule update --init --recursive", gmr_ok)

check("video writer (imageio)",
      lambda: importlib.import_module("imageio") and True,
      "pip install imageio imageio-ffmpeg")

print("=" * 66)
if not problems:
    print("""
    READY.  Nothing else to install.

    Try the real thing now, so nothing surprises you in the session:

    python scripts/xsens_bvh_to_robot_headless.py \\\
    --bvh_file assets/xsens_bvh_test/251021_04_boxing_120Hz_cm_3DsMax.bvh \\\
    --robot unitree_r1 --start 600 --end 1800 --save_path out/mine.pkl

    It should finish in well under a minute and report an IK solve rate above
    100 FPS.  Then:

```

```
python exp4_check_motion.py out/mine.pkl
"""
else:
    print(f"\n NOT READY -- {len(problems)} thing(s) to fix:\n")
    for label, fix in problems:
        print(f"    {label}\n        {fix}\n")
    print("  Full instructions: LAB4_setup_gmr.md")
    print("  Fix these BEFORE the session -- there will not be time during it.\n")
print("=" * 66 + "\n")
sys.exit(1 if problems else 0)
```

```
cd ~/GMR
python ~/r1_lab/exp4/check_setup.py
```

Expected:

```
=====
Lab 4 setup check
=====
[ok] python 3.10+                3.10
[ok] mujoco                      3.6.0
[ok] mink (the IK solver)
[ok] general_motion_retargeting
[ok] R1 robot model              r1_scene.xml
[ok] R1 motion mappings          smplx, bvh_xsens
[ok] the boxing capture          2949 KB
[ok] mujoco_menagerie submodule
[ok] video writer (imageio)
=====

READY.  Nothing else to install.
```

☒ **Bring that verdict to the session.** If it says **NOT READY**, it prints the exact command that fixes each item. The most common cause by far is forgetting **conda activate gmr**.

Anything that fails names its own fix. Work through them and run it again.

The motion file

GMR ships exactly one motion capture, and it is the one this lab uses:

```
assets/xsens_bvh_test/251021_04_boxing_120Hz_cm_3DsMax.bvh
```

3 MB, 4249 frames at 120 Hz = 35 seconds of boxing.

⚠ **The first few seconds are a stationary calibration T-pose, not the capture.** Measure that by accident and you will conclude the retargeting floats the robot 4.5 cm off the ground with a frozen pelvis — all of which is true of the T-pose and false of the motion. Always pass `--start 600 --end 1800`. You will do this wrong **on purpose** in the lab.

The larger public datasets both need free registration, and neither will happen during a session:

dataset	format	works with R1?
AMASS	SMPL-X	☑ via <code>smplx_to_robot.py</code>
LAFAN1	BVH	✗ no R1 config exists

`TEST_MOTIONS.md` in the repo lists which public clips retarget cleanly. Dance and ground-lying motions are the known-hard cases.

If you cannot get it installed

You are not blocked. Ask your instructor for `lab4_fallback.tar.gz` (2.4 MB): a pre-rendered video and the matching motion file, so Parts 3–6 of the lab — including the measurement that is the whole lesson — run without GMR at all.

Only Part 2, the retargeting itself, needs the install.

Troubleshooting

symptom	cause & fix
<code>ModuleNotFoundError: mink</code>	<code>pip install -e .</code> was not run, or ran in the wrong environment
<code>ModuleNotFoundError: general_motion_retargeting</code>	<code>conda activate gmr</code> — by far the most common
missing <code>.xml</code> under <code>mujoco_menagerie/</code>	cloned without <code>--recurse-submodules</code> — run the submodule command
no GMR clone at ... from the installer	your clone is not at <code>~/GMR</code> — pass <code>--gmr /path/to/GMR</code>
no meshes at ... from the installer	Lab 1's folder is gone — pass <code>--meshes /path/to/assets</code>
cannot find the anchor line in <code>params.py</code>	GMR changed upstream. Tell your instructor; the five entries can be added by hand
<code>unitree_r1</code> rejected by <code>--robot</code>	you are on a LAFAN1 script — see the table above
<code>gladLoadGL error</code> on WSL2	the interactive viewer cannot open. Use the <code>_headless</code> script
everything is 50× slower	you passed <code>--video_path</code> . Rendering, not IK, is the bottleneck